Part 3 – Unit Testing Documentation

Project Context

This section presents the **unit testing** carried out for the **Smart Library System**. The tests were designed and executed in an **ad-hoc** manner, focusing on verifying that each individual component works correctly in isolation, prior to integration with other components.

Purpose of Unit Testing

The main goals of unit testing in this project are to:

- Verify the correctness of individual classes and methods
- Detect defects at an early stage of development
- Ensure that each component behaves correctly for both valid and invalid inputs
- Improve code reliability and maintainability

Unit Testing Approach

Unit testing for the Smart Library System was performed by:

- Testing **one class at a time** in isolation
- Replacing dependencies with **mocks or stubs** where required
- Verifying both normal execution paths and exceptional cases
- Avoiding interactions with external components during testing

Test Organization and Responsibilities

To ensure clarity and consistency:

- Unit tests follow the **same package structure** as the main application (API, model, notification, policy, service, and storage)
- Each team member was assigned specific classes to test
- The assignment was based on the **number of test cases** of classes, so that the workload was approximately equal for all members
- Each team member implemented the unit tests for their assigned classes and completed the corresponding documentation tables

Unit Testing Documentation Tables

Class: DefaultLibraryUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_throws_whenCirculationServiceNull()	Checks constructor validation so the facade cannot be created with a null service (prevents NullPointerException later).	Throws IllegalArgumentException with message "CirculationService cannot be null"	Pass
borrowFlow_callsRequestBorrow_onCirculationService()	Confirms borrowFlow() delegates correctly to CirculationService.requestBorrow() with the same parameters.	requestBorrow(1, "BOOK001") called exactly once	Pass
borrowFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalState()	Ensures system handles service failure (IllegalStateException) by wrapping it into a RuntimeException with a clear message.	Throws RuntimeException containing "Failed to complete borrow flow"	Pass
borrowFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalArgument()	Ensures invalid input/arguments from service (IllegalArgumentException) are wrapped into RuntimeException with a clear message.	Throws RuntimeException containing "Failed to complete borrow flow"	Pass
returnFlow_callsReturnItem_onCirculationService()	Confirms returnFlow() delegates correctly to CirculationService.returnItem() with the same item code.	returnItem("BOOK001") called exactly once	Pass
returnFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalState()	Ensures return flow wraps service errors into RuntimeException with clear message.	Throws RuntimeException containing "Failed to complete return flow"	Pass

Class: BorrowRecordUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsFields_andReturnedFalse()	Verify that the BorrowRecord constructor correctly initializes all fields and sets returned to false by default.	All fields set correctly; returned = false; borrow date not null; return date null	Pass
markReturned_setsReturnedTrue()	Verify that markReturned() correctly sets the returned status to true.	returned = true	Pass
markReturned_setsReturnDate()	Verify that markReturned() sets the return date to the current date.	return date equals today's date	Pass
isOverdue_returnsExpected_basedOnDaysAfterDue()	Verify that isOverdue() returns the correct overdue status based on the number of days after the due date.	Correct true/false value returned	Pass
overdueDays_returnsCorrectDays()	Verify that overdueDays() returns the correct number of overdue days when the item is not returned.	Correct overdue day count	Pass
overdueDays_returns0_whenReturned()	Verify that overdueDays() returns 0 when the item has already been returned.	0	Pass
constructor_throws_whenMemberIdInvalid()	Verify that the constructor throws an exception when the member ID is zero or negative.	IllegalArgumentException thrown	Pass
constructor_throws_whenItemCodeInvalid()	Verify that the constructor throws an exception when the item code is empty or blank.	IllegalArgumentException thrown	Pass

constructor_throws_whenItemCodeNull()	Verify that the constructor throws an exception when the item code is null.	IllegalArgumentException thrown	Pass
constructor_throws_whenDueDateNull()	Verify that the constructor throws an exception when the due date is null.	IllegalArgumentException thrown	Pass
equals_returnsTrue_forSameRecord()	Verify that equals() returns true when comparing the same BorrowRecord instance.	true	Pass
equals_returnsFalse_forDifferentRecord()	Verify that equals() returns false when comparing different BorrowRecord objects.	false	Pass

Class: LibraryItemUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsCodeAndAvailability()	Verify that the LibraryItem constructor correctly sets the item code and availability status.	Code = "BOOK001", available = true	Pass
setAvailable_updatesAvailability()	Verify that setAvailable() correctly updates the availability status when set to false and true.	Availability updates correctly	Pass

Class: MembershipTypeUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
membershipType_hasExpectedMaxLoansAndDays()	Verify that each membership type has the correct maximum loans and loan duration values.	Correct max loans and loan days for each type	Pass

Class: MemberUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsIdActiveMembershipType()	Verify that the Member constructor correctly sets the ID, active status, and membership type.	ID = 1, active = true, type = STUDENT	Pass

Class: StubAlertServiceUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
sendAlert_storesLastMemberIdAndMessage()	Verifies the stub stores the last alert data so tests can inspect calls (memberId + message).	After first call: lastMemberId=1, lastMessage="Test message". After second call: lastMemberId=2, lastMessage="Another message"	Pass

Class: DefaultBorrowPolicyUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
canBorrow_returnsFalse_whenMemberIsNull()	Verify that canBorrow() returns false when the member is null.	false	Pass
canBorrow_returnsFalse_whenMemberInactive()	Verify that canBorrow() returns false when the member is inactive.	false	Pass
canBorrow_returnsFalse_whenActiveBorrowsEqualsMax()	Verify that canBorrow() returns false when active borrows equal the membership maximum.	false	Pass
canBorrow_returnsTrue_whenActiveBorrowsLessThanMax_andNoOverdue_andMemberActive()	Verify that canBorrow() returns true when the member is active, has no overdue items, and active borrows are below the limit.	true	Pass
canBorrow_returnsFalse_whenHasOverdueTrue()	Verify that canBorrow() returns false when the member has overdue items.	false	Pass

Class: DefaultPenaltyPolicyUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
calculatePenalty_returnsExpectedAmount()	Verify that calculatePenalty() returns the correct penalty amount based on overdue days, including boundary and capped values.	Correct penalty amount returned	Pass

Class: DefaultCirculationServiceUnitTest

Responsible Members: Viola Zeneli & Sidrit Rrustemi

Test Case	Test Description & Reason	Expected Output	Actual Result
Viola Zeneli			
requestBorrow_throws_whenMemberIdInvalid()	Ensures borrowing is rejected when member ID is zero or negative.	IllegalArgumentException thrown	IllegalArgumentException thrown
requestBorrow_throws_whenItemCodeInvalid()	Verifies borrowing fails when item code is null or empty.	IllegalArgumentException thrown	IllegalArgumentException thrown
requestBorrow_throws_whenMemberNotFound()	Confirms exception is thrown when the member does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenItemNotFound()	Ensures exception when the item does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenItemNotAvailable()	Verifies borrowing is blocked if item is unavailable.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenBorrowPolicyRejects()	Confirms borrowing is denied when policy rules fail.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_updatesAvailabilityToFalse_andSavesBorrowRecord_whenValid()	Ensures availability is updated and record is saved.	Availability updated, record saved	Availability updated, record saved
requestBorrow_savesBorrowRecordWithDueDate_todayPlusMaxLoanDays()	Verifies due date calculation based on membership type.	Correct due date saved	Correct due date saved
returnItem_throws_whenItemCodeNull()	Ensures return fails when item code is invalid.	IllegalArgumentException thrown	IllegalArgumentException thrown
returnItem_throws_whenNoActiveBorrow()	Confirms exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
returnItem_marksRecordReturned_andUpdatesAvailabilityTrue()	Verifies item is returned and availability restored.	Record returned, availability true	Record returned, availability true

returnItem_sendsAlert_whenRecordIsOverdue()	Ensures alert is sent for overdue return.	Alert sent	Alert sent
returnItem_doesNotSendAlert_whenRecordNotOverdue()	Confirms no alert for on-time return.	No alert sent	No alert sent
canBorrowMore_throws_whenMemberNotFound()	Ensures exception when member does not exist.	IllegalStateException thrown	IllegalStateException thrown
canBorrowMore_returnsTrue_whenActiveBorrowsLessThanMax()	Confirms borrowing allowed under capacity.	true	true
canBorrowMore_returnsFalse_whenActiveBorrowsEqualsMax()	Ensures borrowing denied at capacity.	false	false
canBorrowMore_returnsFalse_whenActiveBorrowsGreaterThanMax()	Confirms borrowing denied when capacity exceeded.	false	false
calculateFine_returns0_whenNotOverdue()	Verifies no fine for non-overdue items.	0	0
calculateFine_returnsCorrectAmount_whenOverdue()	Ensures fine is calculated correctly.	Correct fine returned	Correct fine returned
calculateFine_returns0_whenItemReturned()	Confirms no fine for returned items.	0	0
calculateFine_throws_whenNoActiveBorrow()	Ensures exception when no borrow exists.	IllegalStateException thrown	IllegalStateException thrown
calculateFine_throws_whenItemCodeNull()	Ensures invalid item code throws exception.	IllegalArgumentException thrown	IllegalArgumentException thrown
isEligibleToBorrow_returnsTrue_whenAllConditionsMet()	Confirms eligibility when all rules pass.	true	true
isEligibleToBorrow_returnsFalse_whenMemberNull()	Ensures borrowing denied if member missing.	false	false
isEligibleToBorrow_returnsFalse_whenItemNull()	Ensures borrowing denied if item missing.	false	false
isEligibleToBorrow_returnsFalse_whenMemberInactive()	Confirms inactive members cannot borrow.	false	false
isEligibleToBorrow_returnsFalse_whenItemNotAvailable()	Ensures unavailable items cannot be borrowed.	false	false
isEligibleToBorrow_returnsFalse_whenAtCapacity()	Confirms borrowing denied at capacity.	false	false
isEligibleToBorrow_returnsFalse_whenHasOverdue()	Ensures overdue members cannot borrow.	false	false
isEligibleToBorrow_returnsFalse_whenMemberIdInvalid()	Ensures invalid member ID returns false.	false	false

isEligibleToBorrow_returnsFalse_whenItemCodeNull()	Ensures null item code returns false.	false	false
getRemainingBorrowCapacity_returnsCorrectValue()	Verifies correct remaining capacity calculation.	Correct capacity returned	Correct capacity returned
getRemainingBorrowCapacity_throws_whenMemberNotFound()	Ensures exception when member missing.	IllegalStateException thrown	IllegalStateException thrown
validateMemberStatus_returnsExpectedStatus()	Confirms correct status for different member states.	Correct status returned	Correct status returned
validateMemberStatus_returnsInvalid_whenMemberIdNegativeOrZero()	Ensures invalid IDs return error status.	INVALID_MEMBER_ID	INVALID_MEMBER_ID
validateMemberStatus_returnsMEMBER_NOT_FOUND_whenMemberNull()	Confirms MEMBER_NOT_FOUND status.	MEMBER_NOT_FOUND	MEMBER_NOT_FOUND
Sidrit Rrustemi			
getFineCategory_returnsExpectedCategory()	Tests fine categorization based on overdue days.	Correct category (NO_FINE, LOW, MEDIUM, HIGH)	Correct category
getFineCategory_returnsNO_FINE_RETURNED_whenAlreadyReturned()	Verifies no fine category for returned items.	NO_FINE_RETURNED	NO_FINE_RETURNED
getFineCategory_throws_whenNoActiveBorrow()	Ensures exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
canRenewBorrow_returnsTrue_whenAllConditionsMet()	Confirms renewal allowed when all conditions satisfied.	true	true
canRenewBorrow_returnsFalse_whenMemberNotFound()	Verifies renewal denied when member doesn't exist.	false	false
canRenewBorrow_returnsFalse_whenItemNotFound()	Ensures renewal denied when item doesn't exist.	false	false
canRenewBorrow_returnsFalse_whenNoActiveBorrow()	Confirms renewal denied with no active borrow.	false	false
canRenewBorrow_returnsFalse_whenNotTheBorrower()	Verifies renewal denied for non-borrower.	false	false

canRenewBorrow_returnsFalse_whenAlreadyReturned()	Ensures renewal denied for returned items.	false	false
canRenewBorrow_returnsFalse_whenOverdue()	Confirms renewal denied for overdue items.	false	false
canRenewBorrow_returnsFalse_whenMemberInactive()	Verifies renewal denied for inactive members.	false	false
canRenewBorrow_returnsFalse_whenMemberHasOtherOverdueItems()	Ensures renewal denied when member has other overdue items.	false	false
getTotalOverdueDays_returns0_whenNoOverdueItems()	Confirms zero when no items are overdue.	0	0
getTotalOverdueDays_returnsSumOfOverdueDays_whenMultipleOverdueItems()	Verifies correct sum of overdue days across items.	Sum of overdue days	Sum of overdue days
getTotalOverdueDays_ignoresReturnedItems()	Ensures returned items excluded from calculation.	Overdue days excluding returned	Overdue days excluding returned
getTotalOverdueDays_throws_whenMemberIdInvalid()	Verifies exception for invalid member ID.	IllegalArgumentException thrown	IllegalArgumentException thrown
getDaysUntilDue_returnsCorrectDays()	Tests correct calculation of days until due date.	Correct day count	Correct day count
getDaysUntilDue_returns0_whenAlreadyReturned()	Confirms zero for returned items.	0	0
getDaysUntilDue_throws_whenNoActiveBorrow()	Ensures exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
getDaysUntilDue_throws_whenItemCodeNull()	Verifies exception for null item code.	IllegalArgumentException thrown	IllegalArgumentException thrown
calculateLateFeeWithDiscount_appliesDiscountCorrectly()	Tests discount application to late fees.	Discounted fee amount	Discounted fee amount
calculateLateFeeWithDiscount_throws_whenDiscountInvalid()	Ensures exception for invalid discount percentage.	IllegalArgumentException thrown	IllegalArgumentException thrown
getMembershipEligibility_returnsExpectedStatus()	Tests eligibility status for various scenarios.	Expected eligibility status	Expected eligibility status
assessItemReturnStatus_returnsExpectedStatus()	Verifies return status based on days late.	Correct status (EARLY, ON_TIME, LATE, etc.)	Correct status

assessItemReturnStatus_returnsNO_ACTIVE_BORROW_whenNoRecord()	Confirms status when no borrow record exists.	NO_ACTIVE_BORROW	NO_ACTIVE_BORROW
assessItemReturnStatus_returnsALREADY_RETURNED_whenReturned()	Ensures status for already returned items.	ALREADY_RETURNED	ALREADY_RETURNED
canBorrowItemType_respectsValueLimits()	Tests value limits for different membership types.	Correct boolean based on limits	Correct boolean
canBorrowItemType_returnsFalse_whenMemberIdInvalid()	Verifies false for invalid member ID.	false	false
canBorrowItemType_returnsFalse_whenItemTypeNull()	Ensures false for null item type.	false	false
canBorrowItemType_returnsFalse_whenItemValueNegative()	Confirms false for negative item value.	false	false
canBorrowItemType_returnsFalse_whenMemberNotFound()	Verifies false when member doesn't exist.	false	false
canBorrowItemType_returnsFalse_whenMemberInactive()	Ensures false for inactive members.	false	false
canBorrowItemType_returnsFalse_whenNoCapacity()	Confirms false when member at capacity.	false	false
canBorrowItemType_returnsFalse_whenHasOverdue()	Verifies false when member has overdue items.	false	false
shouldSendReminder_returnsExpected()	Tests reminder logic based on days until due and threshold.	Correct boolean	Correct boolean
shouldSendReminder_returnsFalse_whenDaysBeforeDueOutOfRange()	Ensures false for out-of-range reminder days.	false	false
shouldSendReminder_returnsFalse_whenNoActiveBorrow()	Confirms false when no active borrow exists.	false	false
shouldSendReminder_returnsFalse_whenAlreadyReturned()	Verifies false for already returned items.	false	false

Class: MemoryBorrowStorageUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreRecord()	Checks save() stores a borrow record and is counted as active.	findActiveBorrow("BOOK001") not null and countActiveBorrows(1) =1	Pass
findActiveBorrow_shouldReturnMatchingNotReturnedRecord()	Confirms active borrow record is returned for item that is borrowed and not returned.	Returned record not null, itemCode="BOOK001", returned=false	Pass
findActiveBorrow_shouldReturnNullWhenNoMatchingRecord()	Ensures method returns null when no record exists for that item.	findActiveBorrow("NONEXISTENT") returns null	Pass
findActiveBorrow_shouldIgnoreReturnedRecords()	Confirms returned records are not treated as active.	After markReturned(), findActiveBorrow("BOOK001") returns null	Pass
countActiveBorrows_shouldCountCorrectly()	Verifies correct count per member using multiple inputs (parameterized).	Member 1 → 2, Member 2 → 1, Member 3 → 0	Pass
countActiveBorrows_shouldReturnZeroWhenNone()	Ensures 0 is returned when a member has no active borrows.	countActiveBorrows(999) returns 0	Pass
hasOverdueBorrows_shouldReturnTrueWhenMemberHasActiveOverdueBorrow()	Confirms overdue logic works for past due date and still-active borrow.	hasOverdueBorrows(1, today) returns true	Pass
hasOverdueBorrows_shouldReturnFalseWhenMemberHasNoOverdueBorrow()	Confirms future due date is not considered overdue.	hasOverdueBorrows(1, today) returns false	Pass
hasOverdueBorrows_shouldIgnoreReturnedOverdueBorrow()	Ensures overdue records that are already returned are ignored.	After markReturned(), hasOverdueBorrows(1, today) returns false	Pass
findAll_shouldReturnAllSavedRecords()	Confirms retrieving all records returns correct amount.	findAll() list size = 3	Pass
findByMember_returnsOnlyMemberRecords()	Ensures member filtering returns only that member's records.	For member 1: size = 2 and all records have memberId=1	Pass
findByMember_returnsEmptyList_whenNoRecords()	Ensures method returns an empty list (not null) when no records exist for member.	Returned list is not null and is empty	Pass

Class: MemoryItemStorageUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreItem()	Ensures calling save() stores the item and it can be retrieved later.	find("BOOK001") returns the same stored LibraryItem object	Pass
find_shouldReturnItemWhenExists()	Confirms find() returns correct item details when item exists in storage.	Returned item is not null, code="BOOK001", available=true	Pass
find_shouldReturnNullWhenNotExists()	Ensures find() returns null when item code is missing (expected behavior).	find("NONEXISTENT") returns null	Pass
updateAvailability_shouldChangeAvailabilityWhenItemExists()	Confirms updating availability changes stored item state correctly.	After update false → item available=false; after update true → item available=true	Pass
updateAvailability_shouldDoNothingWhenItemNotExists()	Ensures method handles missing items safely (no crash).	No exception thrown when updating availability for missing item	Pass

Class: MemoryMemberStorageUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreMember()	Verify that save() correctly stores a member in memory storage.	Member stored successfully	Pass
find_shouldReturnMemberWhenExists()	Verify that find() returns the correct member when the member exists in storage.	Correct member returned	Pass
find_shouldReturnNullWhenNotExists()	Verify that find() returns null when the member does not exist in storage.	null	Pass